Dinh Thien Tu Tran
Prof Bo Shen
CSCI 611

Style Transfer with VGG19

Reading through the assignment instructions, I understand that the assignment is about using a pretrained model, VGG19, not to classify images, but to extract meaningful visual features from them. We are ignoring the classification part of the network and only using the feature extraction layers. These layers capture different levels of visual information, from simple edges in early layers to complex structures like faces in deeper layers.

Firstly, we need to have a clear picture of VGG19 structure.

```
  (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): ReLU(inplace=True)
  (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (3): ReLU(inplace=True)
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (5): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (6): ReLU(inplace=True)
  (7): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (8): ReLU(inplace=True)
  (9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (10): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (11): ReLU(inplace=True)
  (12): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (13): ReLU(inplace=True)
  (14): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (15): ReLU(inplace=True)
  (16): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (17): ReLU(inplace=True)
  (18): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (19): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (20): ReLU(inplace=True)
  (21): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (22): ReLU(inplace=True)
  (23): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (24): ReLU(inplace=True)
  (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (26): ReLU(inplace=True)
  (27): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (28): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (29): ReLU(inplace=True)
  (30): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (31): ReLU(inplace=True)
  (32): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (33): ReLU(inplace=True)
  (34): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (35): ReLU(inplace=True)
  (36): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
```

Knowing the structure of the model helps implementing functions to get features.

```
## Need the layers for the content and style rep
if layers is None:
    layers = {
        '0' : 'conv1_1',
        '2' : 'conv1_2',
        '5' : 'conv2_1',
        '7' : 'conv2_2',
        '10': 'conv3_1',
        '12': 'conv3_2',
        '14': 'conv3_3',
        '16': 'conv3_4',
        '19': 'conv4_1',
        '21': 'conv4_2',
        '23': 'conv4_3',
        '25': 'conv4_4',
        '28': 'conv5_1',
        '30': 'conv5_2',
        '32': 'conv5_3',
        '34': 'conv5_4'
        }
```

The get_features() function is a really important function, it takes an image, runs through VGG, and saves outputs from certain layers, specifically those from the above image.
When the image goes through the VGG19 model one by one, we will store the output of that layer if it is the one we care about, so now we have snapshots of what the network is working with.

The style of an image is more abstract. It refers to textures, colors, and patterns rather than exact shapes. To capture style, we will have to use multiple layers of VGG that we extracted and compute a Gram matrix for each one.
The Gram matrix tackles an important question:
"How often do different patterns appear together?"
To compute it, we first flatten each feature map into a long vector. Then take the dot product between every pair of channels. This produces a matrix where each entry tells you how similar two channels are, basically saying how strongly those two patterns are correlated across the image.

```python
def gram_matrix(tensor):
    """ Calculate the Gram Matrix of a given tensor
        Gram Matrix: https://en.wikipedia.org/wiki/Gramian_matrix
    """

    ## get the batch_size, depth, height, and width of the Tensor
    ## reshape it, so we're multiplying the features for each channel
    ## calculate the gram matrix

    batch_size, d, h, w = tensor.size()

    features = tensor.view(d, h*w)

    gram = torch.mm(features, features.t())

    return gram
```

In this assignment, three types of loss are used to guide the transformation of the target image. Content loss: measures how closely the generated image preserves the structure of the original content image by comparing their feature representations at a deeper layer of the VGG19 network (conv4_2). The higher this value is, the more similar the generated image is to the style.

Style loss: captures the artistic characteristics of the style image by comparing the Gram matrices of feature maps across several layers. The Gram matrix represents the correlations between different feature channels, effectively encoding texture, color patterns, and brushstroke-like qualities while ignoring spatial arrangement. If style loss is low, the new image has the same texture as the style image, so this needs to be balanced correctly so that the new image does not lose its contents

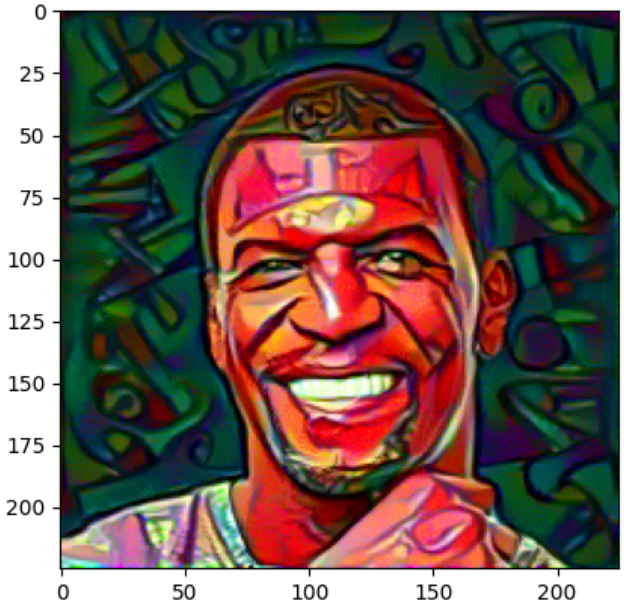
Total loss: the combined value of content and style loss, controlled by hyperparameters content_weight and style_weight, which balance the importance of preserving content versus applying style.

```python
for ii in range(1, steps+1):

    ## get the features from your target image
    ## Then calculate the content loss
    target_features = get_features(target, vgg)
    content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])**2)

    # the style loss
    # initialize the style loss to 0
    style_loss = 0
    # iterate through each style layer and add to the style loss
    for layer in style_weights:
        # get the "target" style representation for the layer
        target_feature = target_features[layer]
        _, d, h, w = target_feature.shape

        ## Calculate the target gram matrix
        target_gram = gram_matrix(target_feature)

        ## get the "style" style representation
        style_gram = style_grams[layer]
        ## Calculate the style loss for one layer, weighted appropriately
        layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**2)

        # add to the style loss
        style_loss += layer_style_loss / (d * h * w)


    ## calculate the *total* loss
    total_loss = content_weight * content_loss + style_weight * style_loss

    ## -- do not need to change code, below -- ##
    # update your target image
    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()
```

TESTING

```python
style_weights = {'conv1_1': 1.,
                 'conv2_1': 0.8,
                 'conv3_1': 0.5,
                 'conv4_1': 0.3,
                 'conv5_1': 0.1}

content_weight = 1  # alpha
style_weight = 1e6  # beta
```

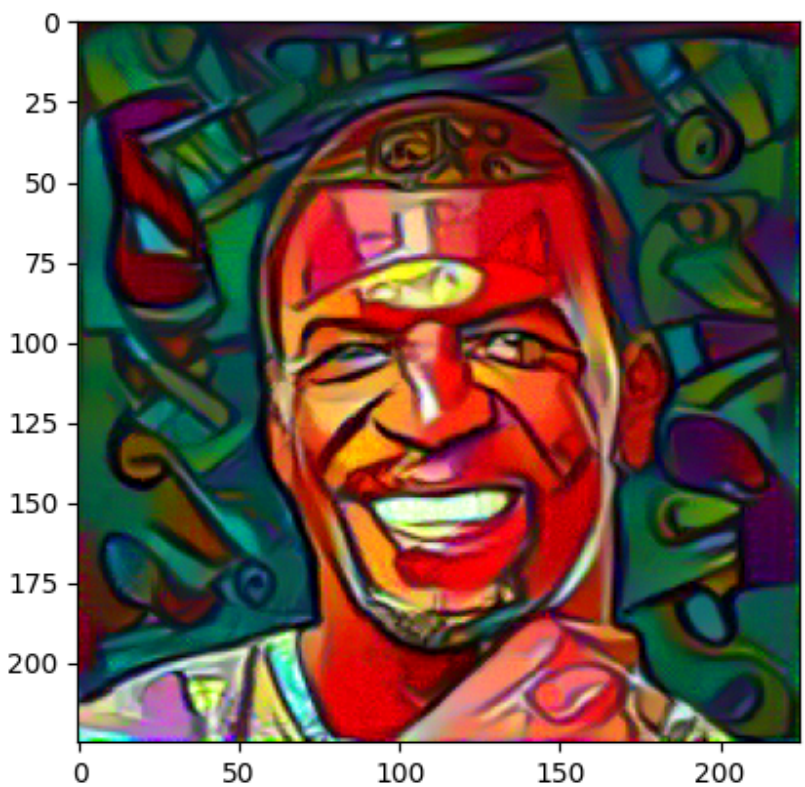
Using the default setting, I achieved



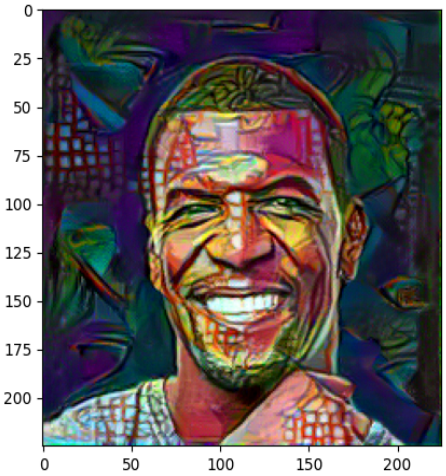
Notice how the result does not inherit the edges like the style image, so I changed some meters. I increased the value of style_weights to get the model to notice strong brushstroke and texture effect. I also decreased beta value to make style more noticeable and increased steps to 4000

```
style_weights = {'conv1_1': 1.,
                 'conv2_1': 1.,
                 'conv3_1': 0.7,
                 'conv4_1': 0.3,
                 'conv5_1': 0.1}

content_weight = .1  # alpha
style_weight = 1e8  # beta

steps = 4000  # decide how many ite

# for displaying the target image,
show_every = 1000
```

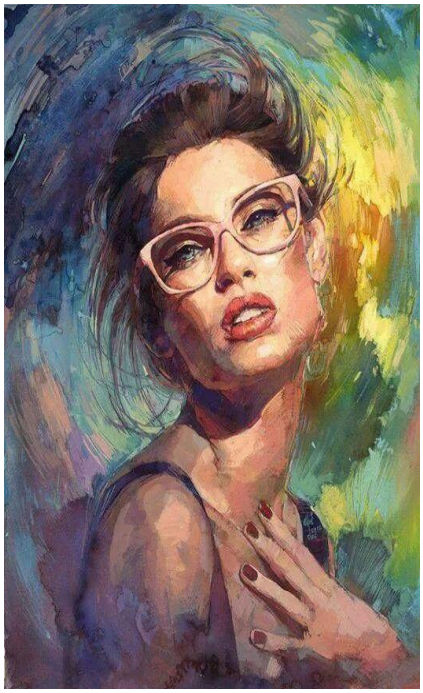
Result:

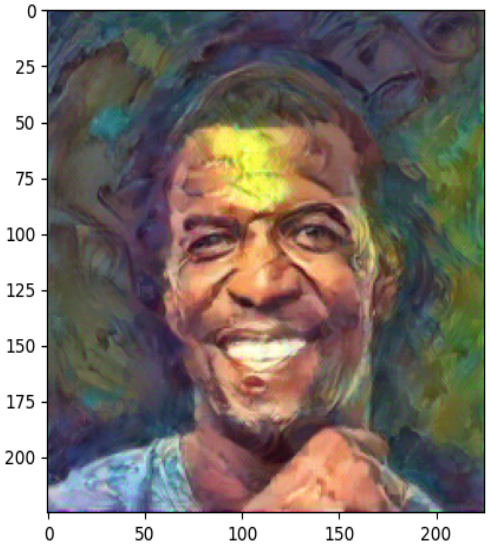


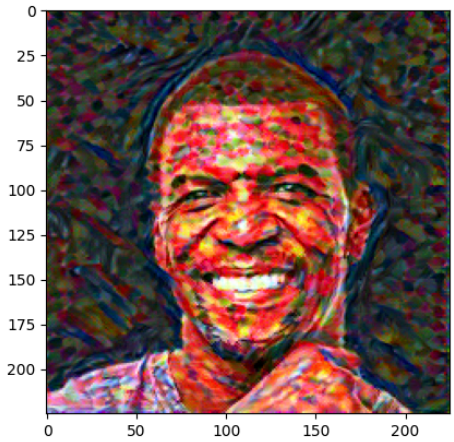
The image does have strong colors, some textures were definitely transferred, but it still looks like the original image. This means the model is not capturing strong artistic structure. I believe this could be due to the style image not being strong enough, so I changed it.

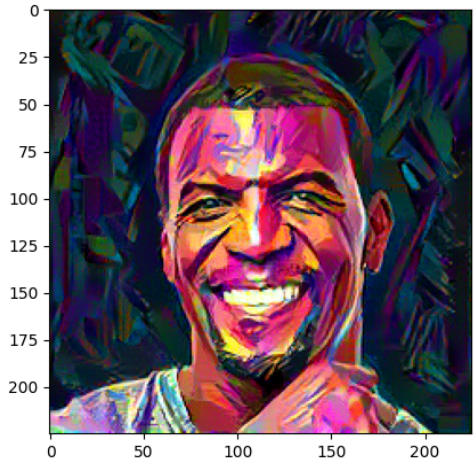


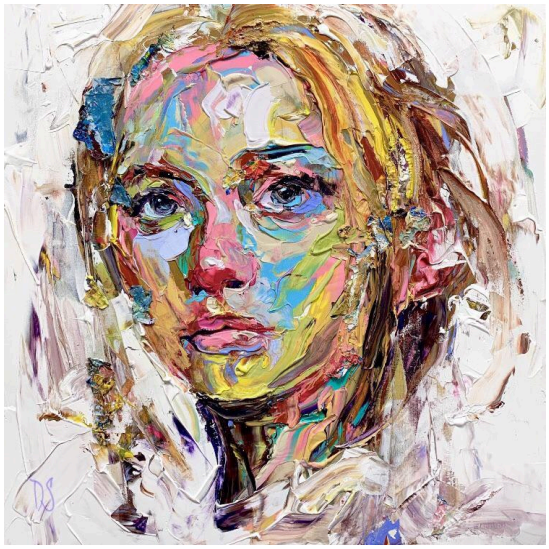
Maybe Picasso's style is too abstract to transfer.
So I changed the style of the image, and kept the default setting, I did do some tests with different parameters but they were not much different.

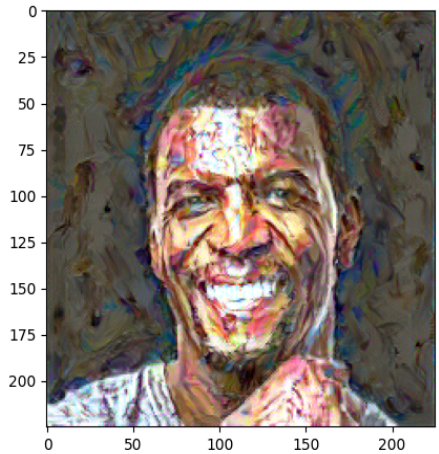





0
25
50
75
100
125
150
175
200
0
50
100
150
200


0
25
50
75
100
125
150
175
200
0
50
100
150
200




0
25
50
75
100
125
150
175
200
0
50
100
150
200